

AI

1. What is Artificial Intelligence?

- a) Artificial Intelligence is a field that aims to make humans more intelligent
- b) Artificial Intelligence is a field that aims to improve the security
- c) Artificial Intelligence is a field that aims to develop intelligent machines
- d) Artificial Intelligence is a field that aims to mine the data

Answer: c

Explanation: Artificial Intelligence is the development of intelligent systems that work and react in the same way that humans do. Intelligence is a process or a component of the ability to achieve goals in the world. People, animals, and a few machines all have different types and degrees of intelligence.

2. The total number of logical symbols in AI are _____

- a) There are 3 logical symbols
- b) There are 5 logical symbols
- c) Number of logical symbols are based on the input
- d) Logical symbols are not used

Answer: b

Explanation: There are totally five logical symbols. The five logical symbols are:

a) Negation

b) Conjunction

c) Disjunction

d) Implication

e) Biconditional

Which of the following is an advantage of artificial intelligence?

- a) Reduces the time taken to solve the problem
- b) Helps in providing security
- c) Have the ability to think hence makes the work easier
- d) All of the above

Answer: d

Explanation: Artificial intelligence creates a machine that can think and make decisions without human involvement.

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

States

Actions

Goal test

Path cost

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

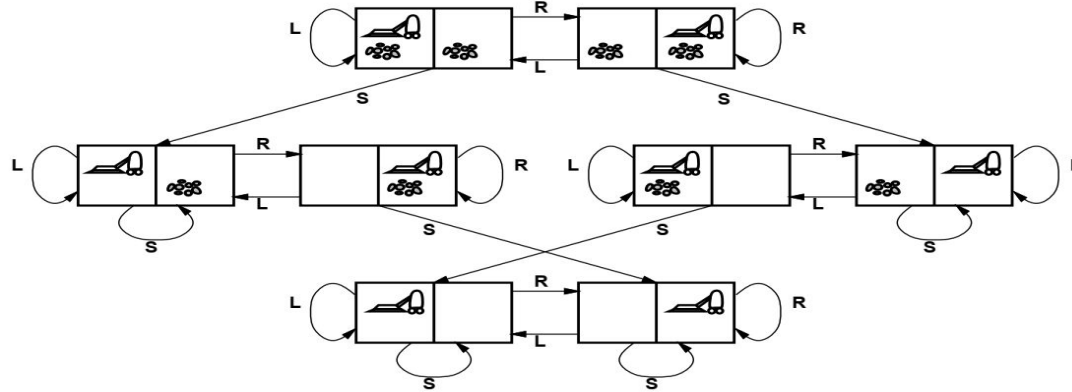
States integer locations of tiles

Actions *left, right, up, down*

Goal test = goal state?

Path cost 1 per move

Example: Vacuum-cleaner

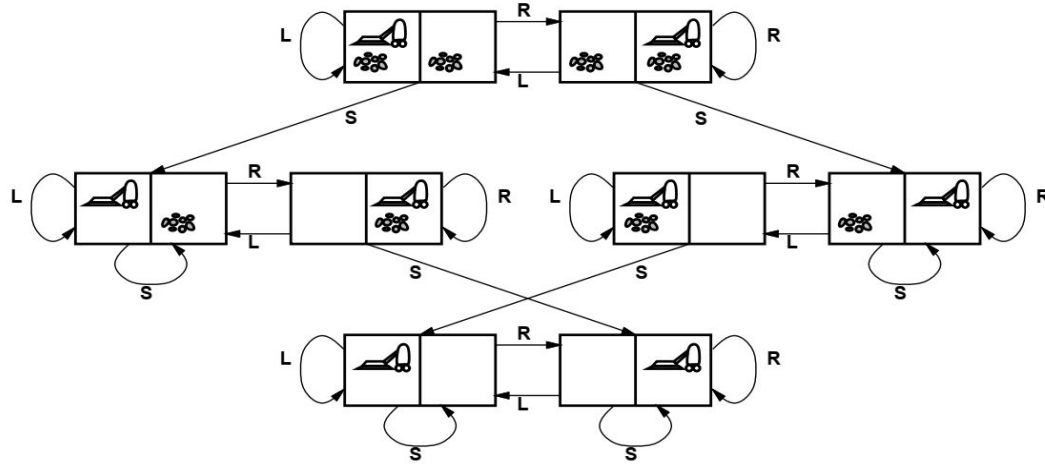


States

Actions

Goal test

Path cost



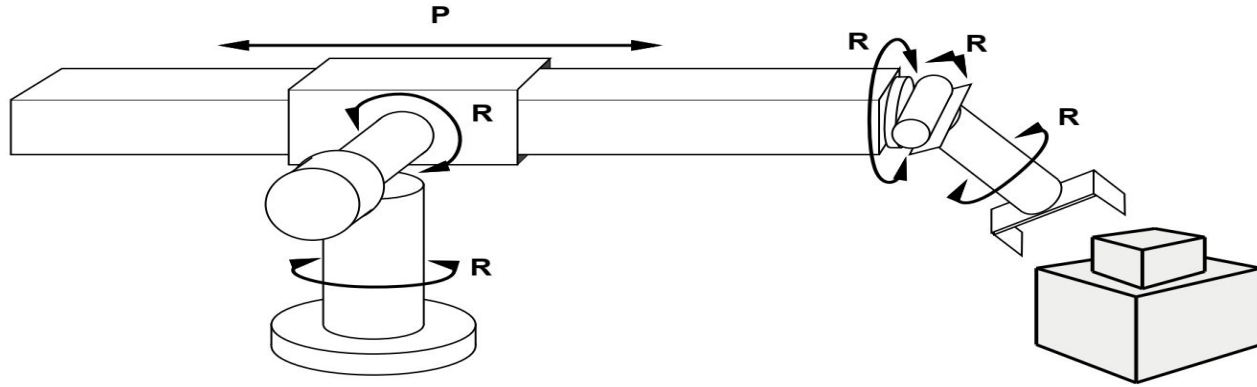
States integer dirt and robot locations

Actions *left, right, suck, noOp*

Goal test *not dirty?*

Path cost 1 per operation (0 for *noOp*)

Example: Robotic assembly



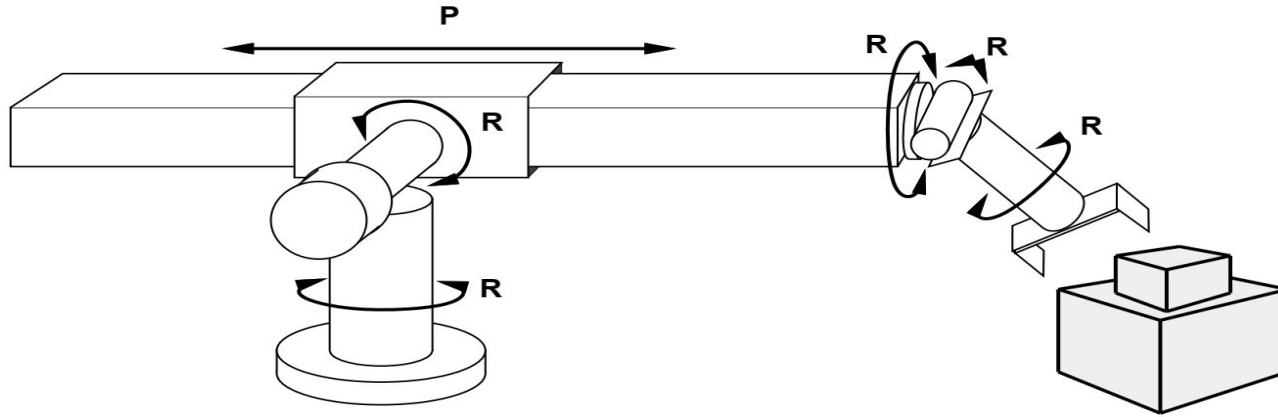
States

Actions

Goal test

Path cost

Example: Robotic assembly



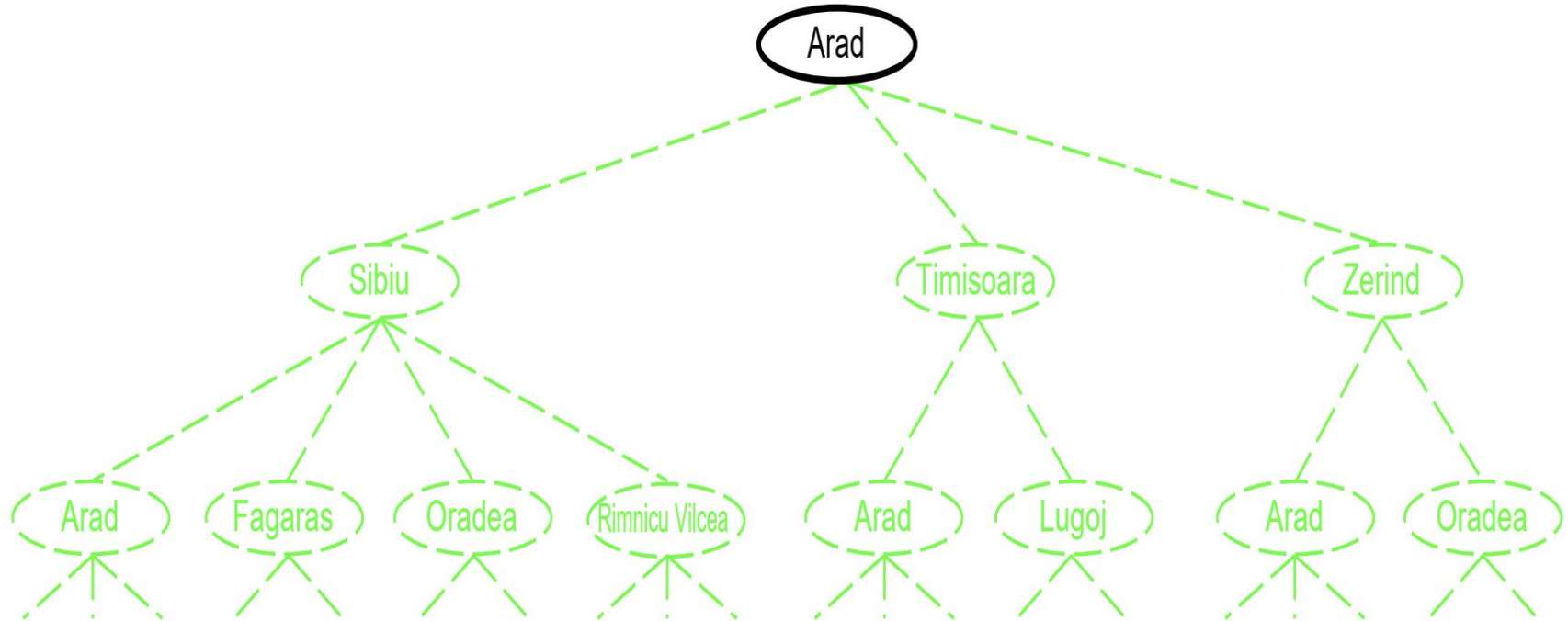
- States** real-valued coordinates of
robot joint angles and parts of the object to be assembled
- Actions** continuous motions of robot joints
- Goal test** assembly complete?
- Path cost** time to execute

Tree search algorithms

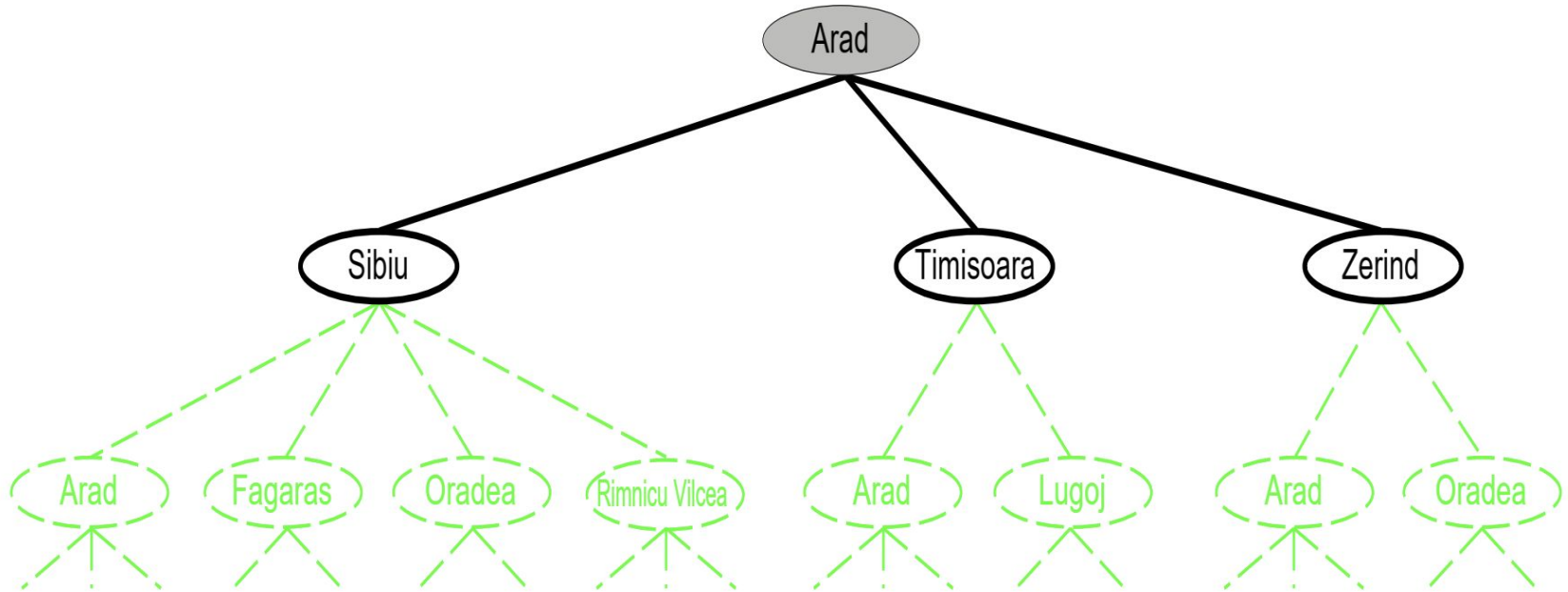
- Offline
- Simulated exploration of state space in a search tree by generating successors of already-explored states

```
function TREE-SEARCH(problem, strategy) returns a solution or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then
      return the corresponding solution
    else
      expand the node and add the resulting nodes to the search tree
  end
```

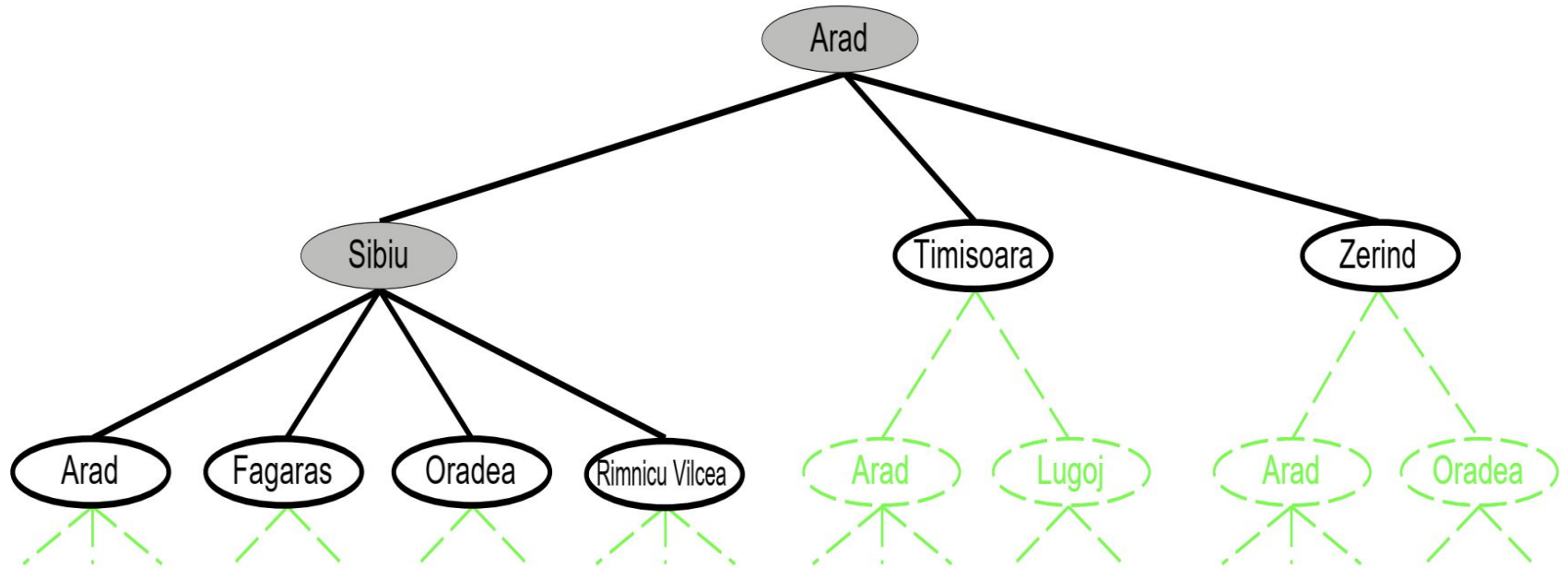
Tree search: Example



Tree search: Example



Tree search: Example



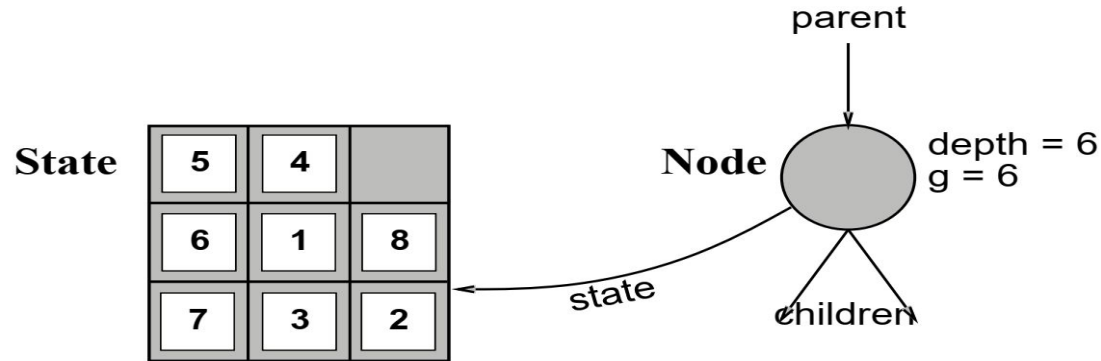
Implementation: States vs. nodes

State

A (representation of) a physical configuration

Node

A data structure constituting part of a search tree
(includes *parent*, *children*, *depth*, *path cost*, etc.)



Implementation of search algorithms

function TREE-SEARCH(*problem*, *fringe*) **returns** a solution or failure

fringe $\leftarrow$ INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*)

loop do

if *fringe* is empty **then return** failure

node $\leftarrow$ REMOVE-FIRST(*fringe*)

if GOAL-TEST[*problem*] applied to STATE(*node*) succeeds **then**

return *node*

else

fringe $\leftarrow$ INSERT-ALL(EXPAND(*node*, *problem*), *fringe*)

end

fringe queue of nodes not yet considered

State gives the state that is represented by *node*

Expand creates new nodes by applying possible actions to *node*

Search strategies

Strategy

Defines the **order** of node expansion

Important properties of strategies

completeness	does it always find a solution if one exists?
time complexity	number of nodes generated/expanded
space complexity	maximum number of nodes in memory
optimality	does it always find a least-cost solution?

Time and space complexity measured in terms of

b	maximum branching factor of the search tree
d	depth of a solution with minimal distance to root
m	maximum depth of the state space (may be ∞)

Uninformed search strategies

Uninformed search

Use only the information available in the problem definition

Frequently used strategies

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

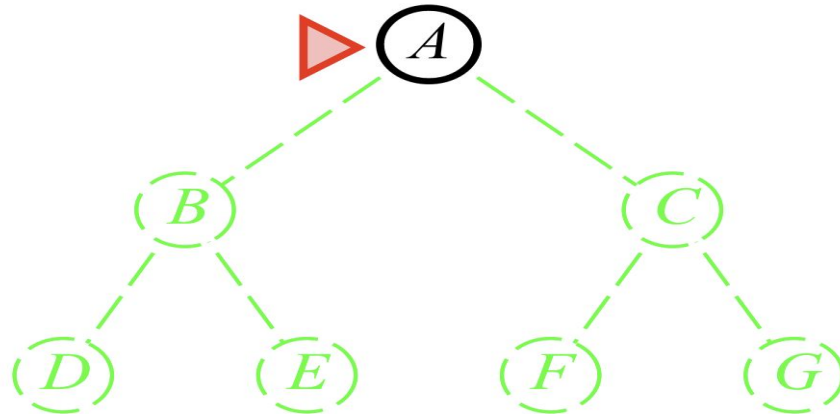
Breadth-first search

Idea

Expand shallowest unexpanded node

Implementation

fringe is a FIFO queue, i.e. successors go in at the end of the queue



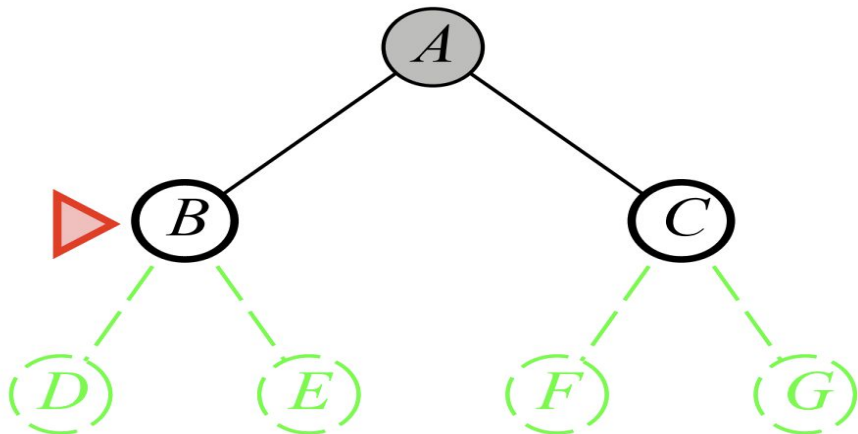
Breadth-first search

Idea

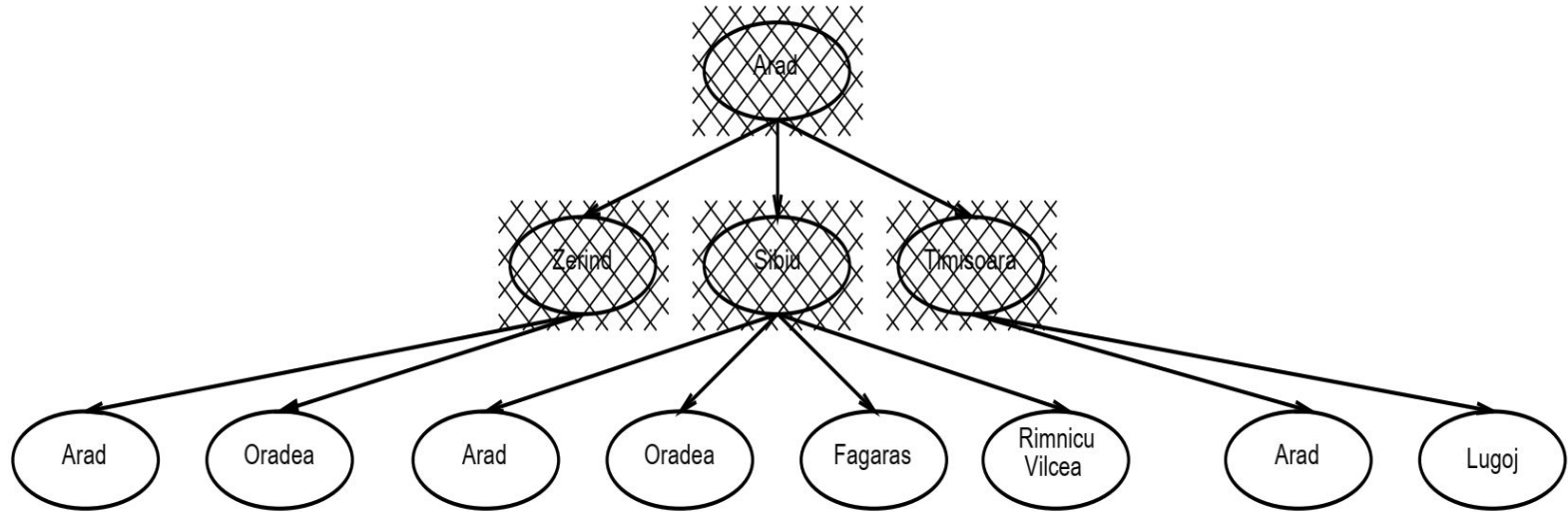
Expand shallowest unexpanded node

Implementation

fringe is a FIFO queue, i.e. successors go in at the end of the queue



Breadth-first search: Example Romania



Breadth-first search: Properties

Complete **Yes** (if b is finite)

Time $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) \in O(b^{d+1})$
i.e. **exponential in d**

Space $O(b^{d+1})$
keeps every node in memory

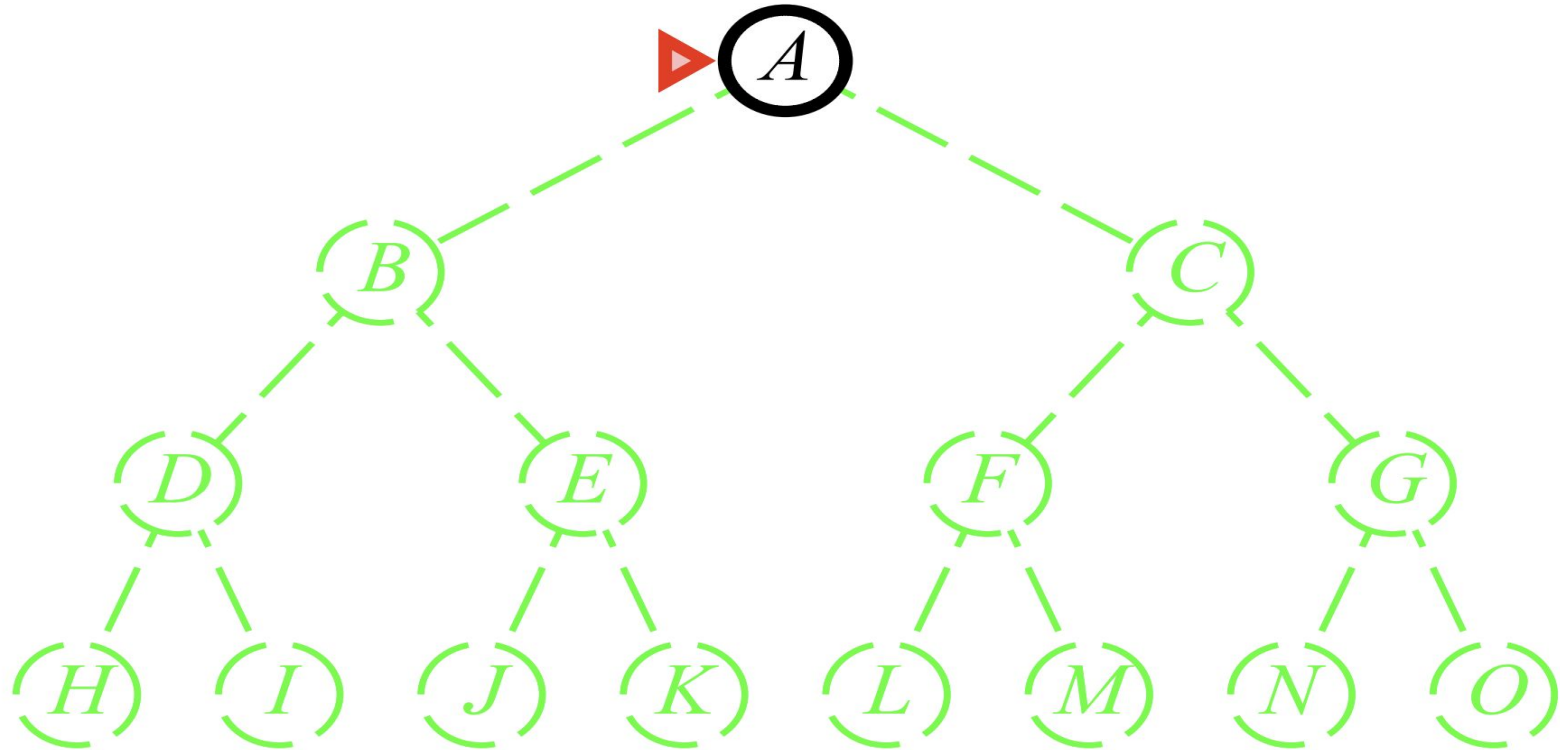
Optimal **Yes (if cost = 1 per step), not optimal in general**

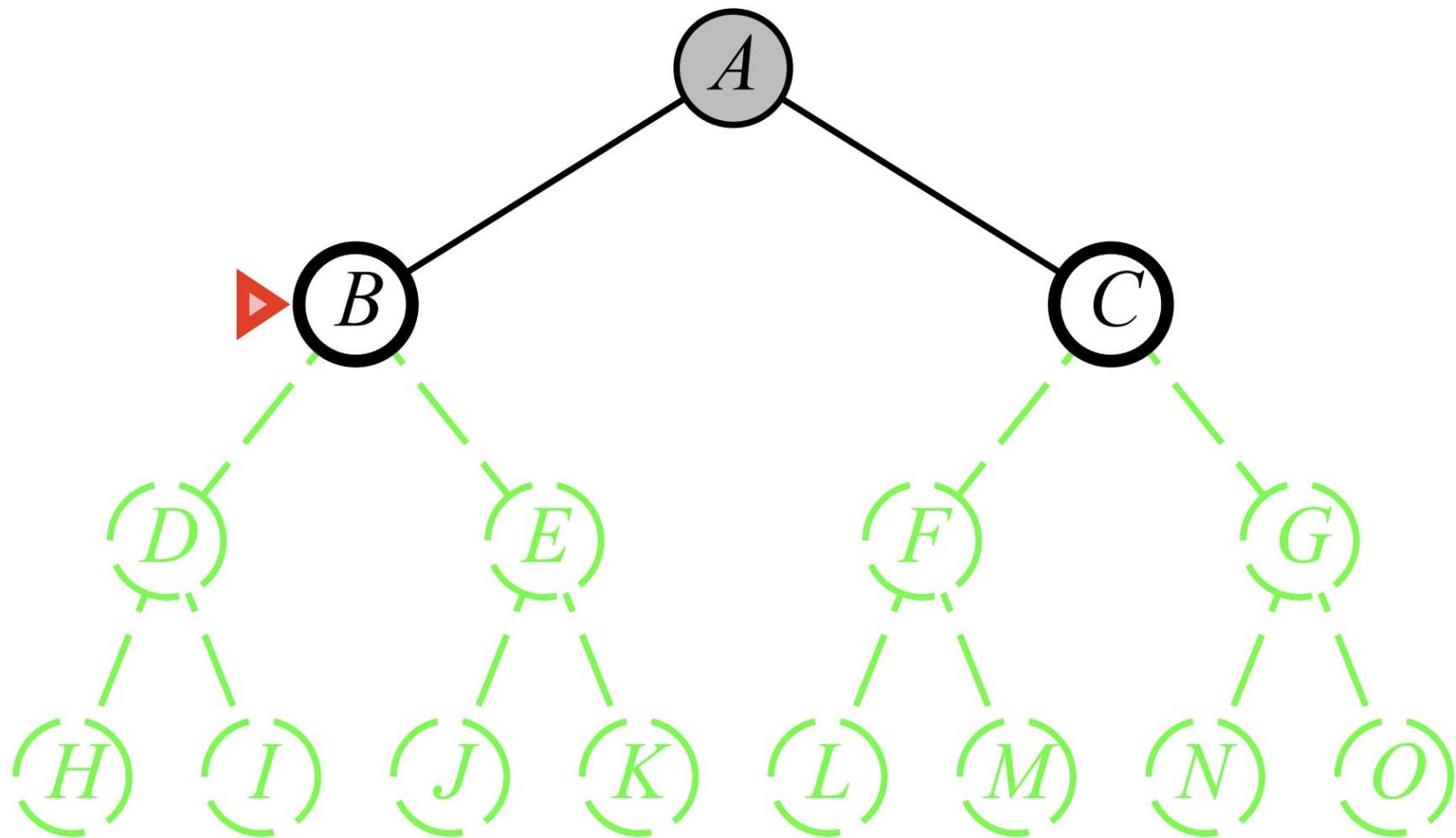
Disadvantage

Space is the big problem

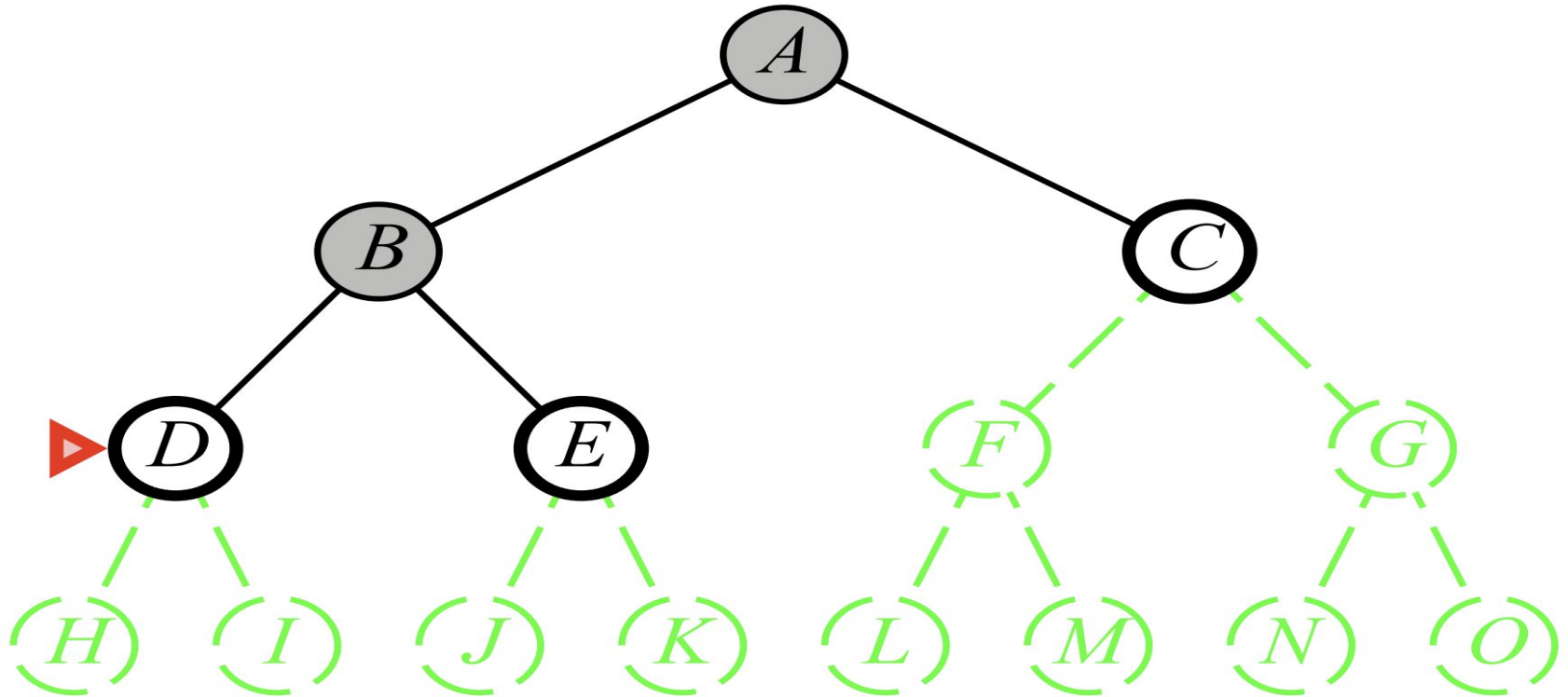
(can easily generate nodes at 5MB/sec so 24hrs = 430GB)

Depth-first search

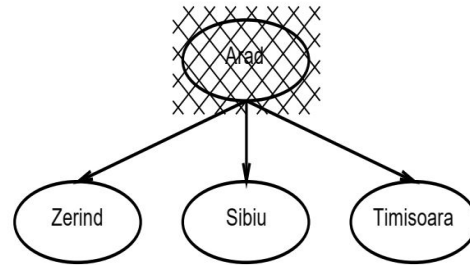




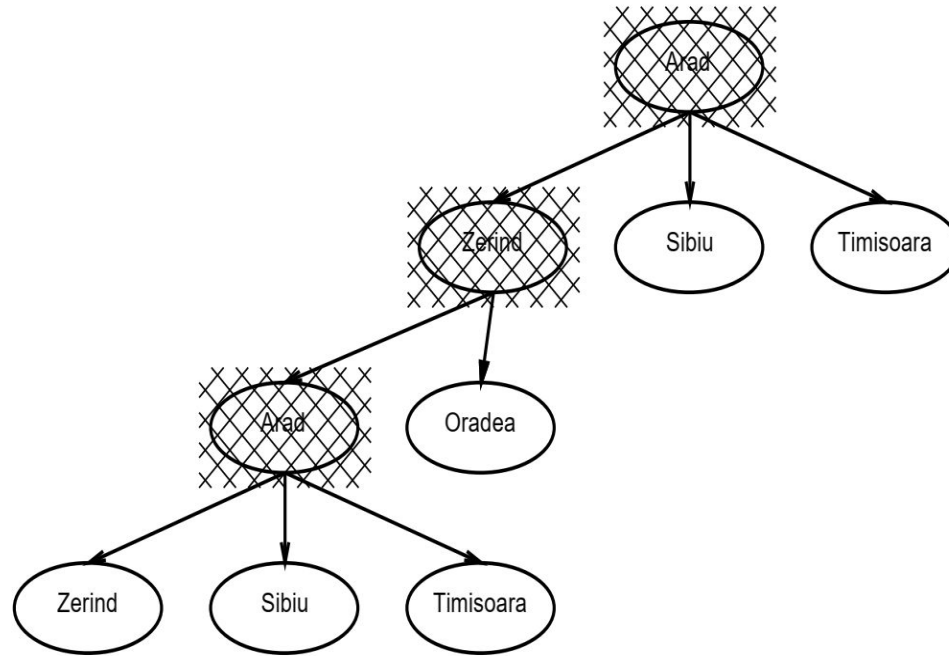
Depth-first search



Depth-first search: Example Romania



Depth-first search: Example Romania



Depth-first search: Properties

Complete	Yes: if state space finite No: if state contains infinite paths or loops
Time	$O(b^m)$
Space	$O(bm)$ (i.e. linear space)
Optimal	No

Disadvantage

Time terrible if m much larger than d

Advantage

Time may be much less than breadth-first search if solutions are dense

Comparison

Criterion	Breadth-first	Uniform-cost	Depth-first	Iterative deepening
Complete?	Yes*	Yes*	No	Yes
Time	b^{d+1}	$\approx b^d$	b^m	b^d
Space	b^{d+1}	$\approx b^d$	bm	bd
Optimal?	Yes*	Yes	No	Yes